

Game Loop & Systems

Kursüberblick

1. Intro
2. Mobile UX & Input
3. Performance & Mobile Constraints
4. GPS & Location-Based Mechanics
5. Map Integration
6. AR Foundation
-  7. Game Loop & Systems
8. Polishing + Smartphone Features
9. Prüfung

Heute

- Creature-Inventory
- Save/Load
- Optimieren der Szenen

Creature Inventory

Creature Inventory

Was definiert eine Creature überhaupt?

- Name
- “Art” (z.B. Mouse-Creature)
- 3D Visuals (Model / Texture)
- 2D Visuals (Icon)
- Basis-Stats
- Spitzname
- Aktuelle Stats
- Gefangen am

Creature Inventory

Was definiert eine Creature überhaupt?

- Name
- "Art" (z.B. Mouse-Creature)
- 3D Visuals (Model / Texture)
- 2D Visuals (Icon)
- Basis-Stats



Allgemeine Werte:

Definition (ScriptableObject)

- Spitzname
- Aktuelle Stats
- Gefangen am



Spezifische Werte:

Runtime Instance

Scriptable Objects

- Unity Objekt für Daten
- Erbt von `Unity.Object`
- Ähnlich wie `MonoBehaviour`
- Sind keine `Components` sondern `Assets`

Nützlich für:

- Speichern von Daten
- Können aber auch Logik enthalten
- Container für “geteilte Daten”, z.B. für einen Monstertyp

<https://docs.unity3d.com/6000.4/Documentation/Manual/class-ScriptableObject.html>

Scriptable Object Erstellen

1. Im Asset Ordner: Create-> Scripting -> ScriptableObject Script
 - a. Im Code Editor öffnen

```
[CreateAssetMenu(fileName = "ScriptableCreature", menuName = "Scriptable Objects/ScriptableCreature")]  
0 references | Unity Script  
public class ScriptableCreature : ScriptableObject
```

- ScriptableCreature erbt von ScriptableObject
- CreateAssetMenu erstellt einen neuen Eintrag im Create Menü:



Scriptable Object befüllen

Für unsere Creature brauchen wir folgende Daten:

- ID (quasi Pokedex ID)
 - wichtig für die Referenzierung
- Name
- "Art" (z.B. Mouse-Creature)
- 3D Visuals (Model / Texture)
- 2D Visuals (Icon)
- Basis-Stats

```
public class ScriptableCreature : ScriptableObject
{
    2 references |  Unity Serialized Field
    public int ID;
    0 references |  Unity Serialized Field
    public string Name;
    0 references |  Unity Serialized Field
    public string Type;
    0 references |  Unity Serialized Field
    public GameObject Model;
    0 references |  Unity Serialized Field
    public Sprite Icon;

    0 references |  Unity Serialized Field
    public int BaseHp;
    0 references |  Unity Serialized Field
    public int BaseAttack;
}
```

Scriptable instanziieren

Das Template steht. Jetzt im Editor im Projekt Ordner:
Create->Scriptable Objects -> Scriptable Creature

Felder befüllen:

Script	ScriptableCreature
ID	1
Name	Bunny
Type	Land
Model	animal-bunny
Icon	BunnySprite
Base Hp	10
Base Attack	1

Runtime Instance

Cool. Der Datentyp existiert. Und was machen wir jetzt damit?

Wir brauchen eine Runtime Instance. Also die Daten für eine einzelne instanziierte Creature.

Unsere **Runtime Instance** ist

- Eine Custom Klasse, die die Creature definiert.
- Nicht von MonoBehaviour abgeleitet
- Nutzt am besten **System.Serializable**, damit wir sie später speichern können

Runtime Instance - Erstellen

1. Neues Script: "CreatureData"
2. Vererbung von MonoBehaviour löschen!
 - a. Wir wollen eine Custom Klasse und kein Objekt vom Typ MonoBehaviour!
3. **System.Serializable** hinzufügen
 - a. Wir wollen, dass die Klasse Serialisiert werden -> also später in ein XML oder Binary-Datenobjekt umgewandelt werden kann.
 - b. Von Klassen die mit System.Serializable markiert sind, kann NICHT geerbt werden

```
[System.Serializable]  
0 references  
public class CreatureData  
{  
  
}
```

Runtime Instance - Befüllen

Für die Runtime Instance brauchen wir:

- Spitzname
- Aktuelle Stats
- Gefangen am

Außerdem Verknüpfung zu den Originaldaten

- ScriptableName
 - Wichtig! Wir wollen hier keine direkte Verlinkung, sondern nur eine indirekte Referenz, da wir später nicht das gesamte ScriptableObject serialisieren wollen.

Runtime Instance - Befüllen II

-

```
[System.Serializable]
0 references
public class CreatureData
{
    //Referenz auf das Feld Name der ScriptableCreature
    0 references
    public int ScriptableID;

    0 references
    public string NickName;
    0 references
    public int CurrentHP;
    0 references
    public DateTime CaughtDateTime;
}
```

Instance und Data verknüpfen

Wir haben das ScriptableObject und die Instance. Jetzt müssen wir sie Ingame verknüpfen:

Dazu bauen wir eine Mini-Datenbank in Unity:

- Enthält eine Liste mit allen ScriptableObjects
- verknüpft diese Liste entsprechend ihren IDs mit den jeweiligen Datenfeldern

Creature-Datenbank erstellen

1. Neues Script. Erbt von MonoBehaviour
2. Singleton (Instance)
3. Datenbank befüllen
4. Funktion um ScriptableCreature aus der Datenbank zu bekommen

Creature-Datenbank Funktionen

In Awake()

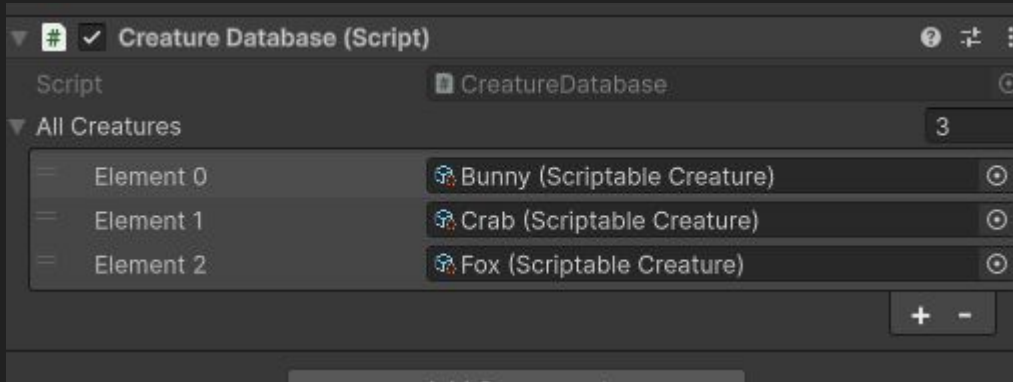
```
// Wir packen die Liste in ein Dictionary für blitzschnellen Zugriff über die ID
_creatureDictionary = new Dictionary<int, ScriptableCreature>();
foreach (var creature in _allCreatures)
{
    if (!_creatureDictionary.ContainsKey(creature.ID))
    {
        _creatureDictionary.Add(creature.ID, creature);
    }
}
```

CreatureSO bekommen:

```
public ScriptableCreature GetPokemonDataByID(int id)
{
    if (_creatureDictionary.TryGetValue(id, out ScriptableCreature data))
    {
        return data;
    }
    Debug.LogError($"Creature mit ID {id} wurde nicht in der Datenbank gefunden!");
    return null;
}
```

Creature-Datenbank befüllen

1. Game Object in Szene erstellen
2. CreatureDatabase hinzufügen
3. Alle Scriptable Objects in die Liste einfügen



Anwendung auf die MapCreature

Okay. Jetzt haben wir alle Voraussetzungen.

Jetzt müssen wir dafür sorgen, dass Creatures die Spawnen auch diese Daten bekommen.

Dafür können wir unsere **MapCreature** erweitern:

- Statt das Visual direkt als Prefab in der MapCreature zu haben, können wir es dynamisch aus dem ScriptableObject laden

MapCreature Erweiterung

1. Neues Feld: `[SerializeField] Transform _visualRoot; //dort wollen wir das Visual erstellen`
2. Neues Feld: `ScriptableCreature _scriptableCreature; //die Creature die uns zugewiesen wird`
3. **SpawnCreature** Funktion erweitern

2 references

```
public void SpawnCreature(Vector2 coordinates, ScriptableCreature scriptableCreatureToSpawn)
{
    // scriptableObject speichern
    _scriptableCreature = scriptableCreatureToSpawn;
    // Visuals instanziiieren und unter die Root hängen
    GameObject creatureVisuals = Instantiate(_scriptableCreature.Model, _visualRoot);
}
```

Spawner umbauen

1. Neue Liste: `[SerializeField] List<ScriptableCreature> _possibleSpawnableCreatures;`
2. Zufällige Creature auswählen und dessen ScriptableObject der **MapCreature** übergeben:

```
int random = Random.Range(0, _possibleSpawnableCreatures.Count);
GameObject newCreature = Instantiate(_creaturePrefab, _parentTransform);
// MapCreature SpawnCreature aufrufen mit Position an der gespawnt werden soll und
// einem zufälligen CreatureType das gespawnt werden soll
newCreature.GetComponent<MapCreature>().SpawnCreature(GetRandomPointAroundPlayer(), _possibleSpawnableCreatures[random]);
```

Prefabs korrigieren

MapCreature:

1. altes Visual löschen. Neue **VisualRoot (Transform)** anlegen und verknüpfen

CreatureSpawner:

1. PossibleSpawnableCreaturesListe befüllen mit Scriptable Objects

CreatureData für die Fang-Szene persitieren

Unsere gespawnten Creatures haben nun Scriptable Objects.

Nun müssen wir diese nur noch in die Fangszene übertragen.

Persistentes Script - GameManager

Dazu legen wir einen GameManager an, der durch alle Szenen persistiert!

1. Script erstellen: **GameManager**
2. Script Singleton machen (`public static GameManager Instance;`)
3. Script DontDestroyOnLoad machen(`DontDestroyOnLoad(this.gameObject);`)
4. Neues Feld: `public ScriptableCreature SelectedCreature {get; private set;}`
5. Neue Funktionen:

```
public void SetSelectedCreature(ScriptableCreature scriptableCreature)
{
    SelectedCreature = scriptableCreature;
}
0 references
public void UnSelectCreature()
{
    SelectedCreature = null;
}
```

GameManager in Szene erstellen und aufrufen

1. Neues GameObject in MapSzene -> GameplayManager drauf
2. In **MapCreature** bevor wir die Szene wechseln, **GameplayManager** aufrufen und ihm unsere aktuelle Creature als **SelectedCreature** übergeben

```
GameplayManager.Instance.SetSelectedCreature(_scriptableCreature);
```

3. Anschließend CatchSzene normal laden

Gewählte Creature in Catch Szene laden

1. In **ARCreatureSpawner** die ARCreature erweitern
 - a. Zunächst auch wieder das “allgemeine” Visual entfernen
 - b. dann, wie in MapCreature in **CreatureLogic** ein neues Feld für
`[SerializeField] Transform _visualRoot;` und
`ScriptableCreature _scriptableCreature;`
 - c. Neue Funktion:

```
public void SetCreatureType(ScriptableCreature scriptableCreature)
{
    _scriptableCreature = scriptableCreature;
    GameObject creatureVisuals = Instantiate(_scriptableCreature.Model, _visualRoot);
}
```

Gewählte Creature in Catch Szene laden II

1. Setzen der Creature im **ARCreatureSpawner**:

```
// 1. Die Kreatur ganz normal in die Welt setzen
spawnedCreature = Instantiate(creaturePrefab, spawnPosition, spawnRotation);
spawnedCreature.GetComponent<CreatureLogic>().SetCreatureType(GameplayManager.Instance.SelectedCreature);
```

2. Am besten noch vorher Null Checks machen:

```
private void SpawnProcess()
{
    //early out if to prevent null refs
    if (GameplayManager.Instance == null)
    {
        Debug.LogError("Gameplay Manager is null");
        return;
    }

    if (GameplayManager.Instance.SelectedCreature == null)
    {
        Debug.LogError("Selected Creature is null");
        return;
    }
}
```

Gewählte Creature in Catch Szene laden III

- Verknüpfungen in den Prefabs setzen nicht vergessen!

Dann testen!

Kurze Pause?

Player Inventory

Creature Fangen und im “Inventar” ablegen

Damit die gefangen Creatures gespeichert werden können, müssen wir sie beim Fangen im Inventar ablegen.

Dazu müssen wir

- ein Inventar anlegen
- beim Fangen ein “Creature-Data” Object erstellen
- das “Creature-Data” Objekt befüllen
- das “Creature-Data” Objekt im Inventar ablegen

Inventar anlegen

1. Im GameManager eine Liste aus CreatureData anlegen
2. Eine Funktion hinzufügen, um die gefangene Creature hinzuzufügen

```
public List<CreatureData> CaughtCreatures = new List<CreatureData>();
```

```
public void AddCaughtCreature(CreatureData caughtCreature)
{
    CaughtCreatures.Add(caughtCreature);
}
```

CreatureData erzeugen

1. Beim Fangen **CreatureData** anlegen
2. **CreatureData** befüllen:
 - a. mindestens mit ID
 - b. (optional) CaughtDateTime
3. Die befüllte **CreatureData** dem GameManager übergeben

CreatureData erzeugen - Beispielcode

```
1 reference
void CatchLogic()
{
    isMoving = false; // Bewegung stoppen
    ARSceneManager.Instance.StartEndRoutine();

    CreatureData caughtCreature = new CreatureData();
    // ID zuweisen
    caughtCreature.ScriptableID = _scriptableCreature.ID;
    // Fangzeitpunkt speichern
    caughtCreature.CaughtDateTime = DateTime.Now;

    // Creature Data in Inventar übertragen
    GameManager.Instance.AddCaughtCreature(caughtCreature);

    Invoke("DisableSelf", 0.9f);
}
```

Testen &
Kurze Pause?

Save/Load System

Save/Load Optionen

- in PlayerPrefs
- in lokaler Datei
- lokale Datenbank (z.B) SQLite
- in Cloud

-> einfachste Variante: lokale Datei

Creatures in Datei speichern

Um die gefangenen Creatures in einer Datei zu speichern, müssen wir eigentlich nur unsere `List<CreatureData>` abspeichern.

Am einfachsten, in dem wir ein JSON aus unserer befüllten Liste generieren.

Dazu können wir die `JsonUtility` von Unity nutzen.

SaveManager Script - DataWrapper

1. Neues Script anlegen: "SaveManager",
2. Zuerst brauchen wir einen DataWrapper

```
[System.Serializable]
4 references
public class SaveDataWrapper
{
    2 references
    public List<CreatureData> CaughtCreatures = new List<CreatureData>();
}
```

Was macht der?

Der packt unsere Liste in eine eigene Klasse, damit wir die gesamte Klasse als einzelnes Objekt speichern können

SaveManager Script - SaveManager

Klasse selbst:

```
0 references
public static class SaveManager
{
    4 references
    private static string SavePath => Path.Combine(Application.persistentDataPath, "savegame.json");
```

SaveManager Script - Save Function

Save Funktion:

```
// SPEICHERN
0 references
public static void SaveCreatures(List<CreatureData> creaturesToSave)
{
    try
    {
        // Daten in den Wrapper packen
        SaveDataWrapper wrapper = new SaveDataWrapper();
        wrapper.CaughtCreatures = creaturesToSave;

        // In JSON-String umwandeln (true für "prettyPrint", damit man es am PC lesen kann)
        string json = JsonUtility.ToJson(wrapper, true);

        // Datei auf Android schreiben
        File.WriteAllText(SavePath, json);

        Debug.Log($"Erfolgreich gespeichert unter: {SavePath}");
    }
    catch (Exception e)
    {
        Debug.LogError($"Fehler beim Speichern: {e.Message}");
    }
}
```

SaveManager Script - Load Function

Load Funktion:

```
public static List<CreatureData> LoadCreatures()
{
    // Wenn noch keine Datei existiert, gib eine leere Liste zurück
    if (!File.Exists(SavePath))
    {
        Debug.Log("Kein Savegame gefunden. Starte mit leerer Liste.");
        return new List<CreatureData>();
    }

    try
    {
        // JSON-Text aus der Datei lesen
        string json = File.ReadAllText(SavePath);

        // Aus dem JSON wieder das Wrapper-Objekt machen
        SaveDataWrapper wrapper = JsonUtility.FromJson<SaveDataWrapper>(json);

        Debug.Log("Savegame erfolgreich geladen.");
        return wrapper.CaughtCreatures;
    }
    catch (Exception e)
    {
        Debug.LogError($"Fehler beim Laden: {e.Message}");
        return new List<CreatureData>();
    }
}
```

Benutzen des SaveManagers

1. Um die Creature List zu speichern, einfach: `SaveManager.SaveCreatures(CaughtCreatures);`

z.B.: direkt in `AddCaughtCreature()` im **GameManager**

2. Um die Creature List zu laden, einfach:

```
CaughtCreatures = SaveManager.LoadCreatures();
```

z.B.: direkt in `Start()` im **GameManager**

3. *Optional*: Löschen des Speicherstands über: `File.Delete(SavePath);`

Kurze Pause?

Testen &
Kurze Pause?

Creature Viewer aus Inventar

Creature Viewer aus Inventar

1. Creature Viewer Scene anpassen, statt festen Creatures, die erste Creature der Liste laden
 - a. wenn keine Creature gefangen sind, Platzhalter oder Text anzeigen
2. Zwei Buttons, einen für “vorherige” einen für “nächste” Creature
 - a. zum Durchschalten der Liste
3. Ein neues Script für die Durchschaltlogik: **CreatureSelector**

CreatureSelector

1. In **Start()**: Prüfen, ob der Spieler überhaupt schon Monster gefangen hat
 - a. wenn ja: -> ShowCreature an Index 0
 - b. wenn nein: -> Default Text anzeigen
2. **ShowCreature(int index)**
 - a. wenn wir gerade ein Model anzeigen -> dieses zerstören
 - b. das Data Objekt mit Wert **index** aus der **List** im GameManager holen
 - c. per Lookup in unserer **CreatureDatabase** anhand der **ID** im DataObjekt die passende **ScriptableCreature** finden
 - d. Aus der gefundenen **ScriptableCreature** das Model extrahieren und an dem ParentTransform instanziiieren.
3. Funktion für NextButtonClicked und PreviousButtonClicked
 - a. index hochzählen. Wenn Index größer als List.Count -> Index = 0

CreatureSelector - Beispiellösung

Variablen:

```
private int _currentIndex = 0;           // Das aktuell angezeigte Monster  
no commands  
private GameObject _currentModel;       // Referenz auf das gerade aktive 3D-Modell  
no commands  
[SerializeField] private Transform _spawnPoint;
```

```
void Start()  
{  
    if (GameManager.Instance.CaughtCreatures.Count > 0)  
    {  
        _currentIndex = 0;  
        ShowCreature(_currentIndex);  
    }  
    else  
    {  
        // Text oder Icon für "keine Creature gefangen"  
        Debug.Log("Inventar ist leer, nichts zum Anzeigen da.");  
    }  
}
```

CreatureSelector - Beispiellösung II

```
private void ShowCreature(int index)
{
    // Sicherheits-Check
    if (index < 0 || index >= GameManager.Instance.CaughtCreatures.Count) return;

    // 1. Altes Modell zerstören, falls eins da ist
    if (_currentModel != null)
    {
        Destroy(_currentModel);
    }

    // 2. Die aktuellen Daten aus der Liste holen
    CreatureData currentData = GameManager.Instance.CaughtCreatures[index];

    // 3. Den Lookup in der Datenbank machen (Die Brücke zum ScriptableObject)
    ScriptableCreature scriptableCreature = CreatureDatabase.Instance.GetCreatureByID(currentData.ScriptableID);

    if (scriptableCreature != null && scriptableCreature.Model != null)
    {
        // 4. Neues Modell am Spawnpoint instanziiieren
        _currentModel = Instantiate(scriptableCreature.Model, _spawnPoint.position, _spawnPoint.rotation, _spawnPoint);
    }
}
```

CreatureSelector - Beispiellösung III

NextButtonClicked:

```
public void OnNextButtonClicked()
{
    if (GameManager.Instance.CaughtCreatures.Count == 0) return;

    _currentIndex++;

    // Loop-Effekt: Wenn wir am Ende sind, springe wieder zum ersten Monster (Index 0)
    if (_currentIndex >= GameManager.Instance.CaughtCreatures.Count)
    {
        _currentIndex = 0;
    }

    ShowCreature(_currentIndex);
}
```

CreatureSelector verknüpfen und testen

1. Button Click Events mit NextButtonClicked und PreviousButtonClicked verknüpfen
2. Evtl. SelectCreature aus dem
3. Creature fangen und Inventar testen

Scene Optimization

Scene Struktur

1. Menu Scene
2. Map Scene
3. Creature Viewer (Creature Inventory Viewer)
4. Capture Scene (AR)

Problem: Laden der Map Scene

- Die Map Scene ist unsere “Basis-Szene”. Wir brauchen sie ständig.
- Aber sie lädt langsam, da die virtuelle Map jedesmal neu aufgebaut werden muss
- Was für Optionen haben wir?

1. Pre-loading

- a. wird vermutlich nicht viel bringen, da die Map erst beim Öffnen initialisiert wird

2. Offen halten der Map Scene

- a. müssen aber Cesium “smart” deaktivieren, damit er nicht “neu intialisiert”

Async Loading

1. Was ist Asynchronous Loading?
2. Wofür können wir das nutzen?

Verbessern der Szene

1. Creatures auf der Map persistieren
 - a. Also über Szenenwechsel hinweg.
 - b. Gefangene Creatures müssen despawnen
 - c. Nicht gefangene Creatures despawnen nach einer bestimmten Zeit